

MALLA REDDY TECHNICAL CAMPUS

A constituent Unit of MALLA REDDY VISHWAVIDYAPEETH

Deemed to be University

Approved by UGC & AICTE New Delhi

Maisammaguda, Medchal (Dist), Hyderabad – 500100, Telangana



TEAM MEMBERS:

DEVIJ BHAGYASREE~ 25MVCSER0030

JUKANTI BHAVYA~ 25MVCSER0051

NARI SREEJA ~ 25MVCSER0066

POCHAMAINA AKSHITA ~ 25MVCSER0072

BRANCH:CSE~A

PROBLEM STATEMENT : SMART INVENTORY MANAGEMENT

SELECT THE RIGHT PRODUCTS UNDER LIMITED STORAGE CAPACITY

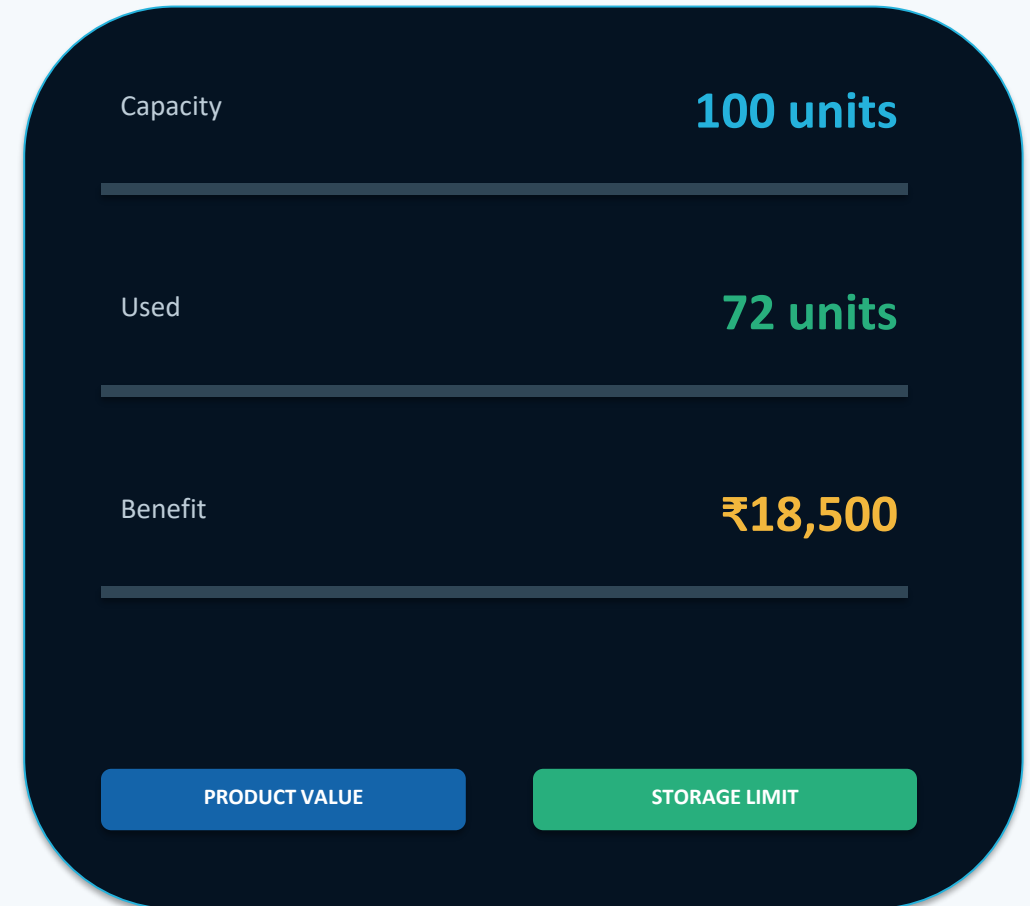
Smart Inventory Optimization System

DAA • Dynamic Programming • 0/1 Knapsack

● Problem Statement

Select products to stock under limited storage capacity while maximizing expected benefit.

The system must decide which products to keep, without exceeding warehouse capacity.



FROM STORAGE CONSTRAINTS TO DATA-DRIVEN STOCK SELECTION

Real-world Issue

- Warehouse space is limited and costly.
- Stocking every product is not possible.
- High-value products may consume more space.
- Manual selection can miss the best combination.
- Poor allocation can reduce expected return.

1 INPUT DATA

Product name • storage • expected benefit



2 OPTIMIZATION ENGINE

0/1 Knapsack + Dynamic Programming



3 BEST COMBINATION

Choose items within capacity

Solution

- Enter products and warehouse capacity.
- Build a DP table for all capacity values.
- Compare selecting vs. skipping each product.
- Recover the selected products by backtracking.
- Display maximum benefit and storage usage.

METHODOLOGY • ALGORITHM • IMPLEMENTATION

Methodology & Process

- Collect product details.
- Validate positive storage and benefit values.
- Set maximum warehouse capacity.
- Create DP table $dp[i][w]$.
- Compute best benefit for every state.
- Backtrack to identify selected products.
- Show utilization, benefit and recommendation.

0/1 KNAPSACK DP

Product i

Current item

Capacity w

Available space

Skip

$dp[i-1][w]$

Take

$benefit[i] + dp[i-1][w-space[i]]$

Choose MAX

Best feasible benefit

$dp[i][w] = \max(skip, take)$

Technology Stack

- HTML5 + CSS3
- JavaScript
- Dynamic Programming
- Responsive UI

Complexity

- Time: $O(n \times W)$
- Space: $O(n \times W)$
- n = number of products
- W = storage capacity

PRACTICAL • SCALABLE • USEFUL

Feasibility

- Works with standard web technologies.
- No specialized hardware is required for the prototype.
- Algorithm is well suited to constrained inventory selection.
- Can run locally through a browser.
- Can be connected to databases or APIs later.

Viability

- Helps warehouses use limited space efficiently.
- Provides an explainable recommendation.
- Easy for staff to understand: space vs. benefit.
- Can support multiple product categories.
- Can be extended with demand forecasts.

Business Potential

- Reduce low-value space allocation.
- Improve expected inventory return.
- Support faster stocking decisions.
- Create a foundation for smart warehouse tools.
- Future: alerts, analytics and real-time inventory.

EXPECTED VALUE OF SMART STOCK SELECTION

1 Space Efficiency

Uses available storage within the limit.

2 Higher Benefit

Selects the combination with maximum expected return.

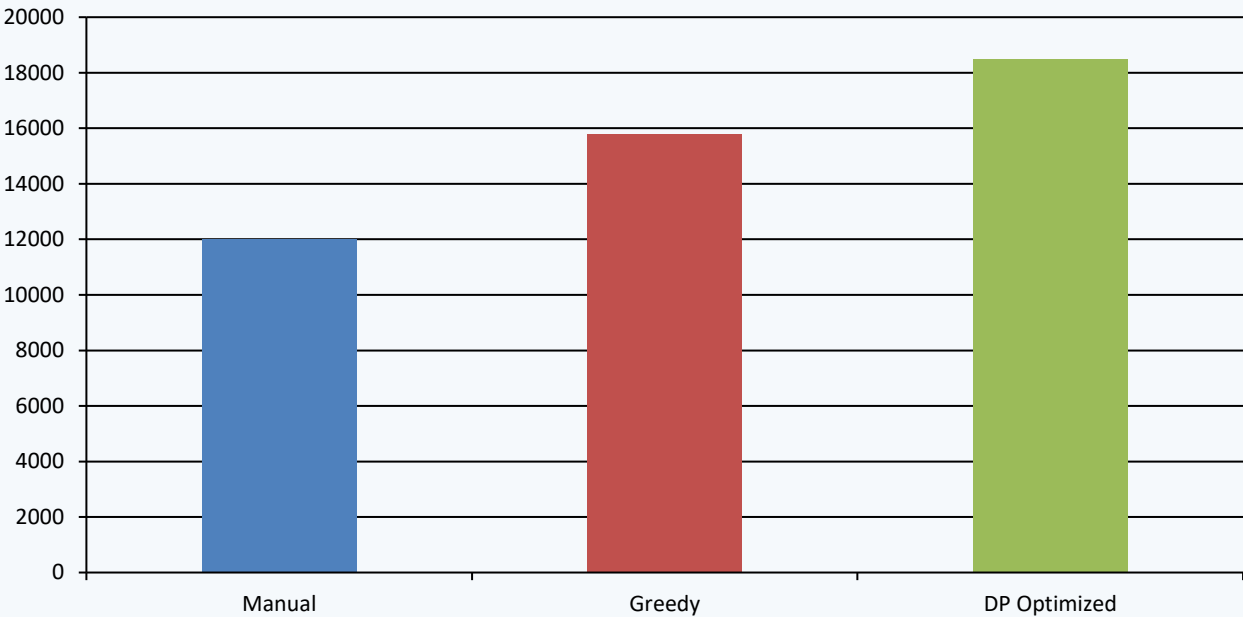
3 Faster Decisions

Automates repetitive product-selection work.

4 Transparency

Shows why products were selected.

Illustrative Benefit Comparison



Storage constraint
Respected

Decision time
Reduced

Selection
Explainable

VALIDATION • EXTENSION • DEPLOYMENT

References / Concepts

- 0/1 Knapsack Problem
- Dynamic Programming principles
- Algorithm Design & Analysis
- Warehouse / inventory optimization concepts
- Web-based dashboard design

SMART INVENTORY DASHBOARD

Laptop

5 units ₹5000

Smartphone

3 units ₹3000

Watch

2 units ₹2500

75% STORAGE UTILIZED

Future Scope

- Real-time inventory database
- Demand prediction using ML
- Expiry and reorder alerts
- Multi-warehouse optimization
- User login and role-based access
- Cloud deployment and analytics

“Store smarter — maximize value from every unit of space.”



*Thank
You!*

Thanks for trusting us. Let's push boundaries together